Class Activity — 15.07.2026

Color Scales & Coordinate Systems: Python + R, Code & Output, All 10 Scenarios

Each scenario: Python code + output, followed by R code + output

Generated: 15 July 2026

Scenario 1

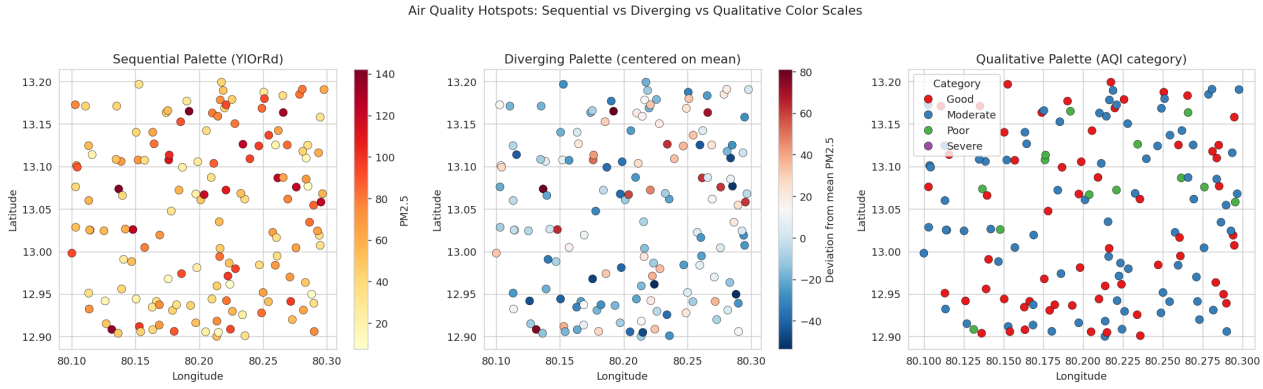
Smart City Air Quality Monitoring

PYTHON

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_style("whitegrid")
np.random.seed(1)

n = 150
lat = np.random.uniform(12.9, 13.2, n)
lon = np.random.uniform(80.1, 80.3, n)
pm25 = np.random.gamma(4, 15, n)
df1 = pd.DataFrame({'lat': lat, 'lon': lon, 'PM2.5': pm25})

fig, axes = plt.subplots(1, 3, figsize=(17, 5))

sc0 = axes[0].scatter(df1.lon, df1.lat, c=df1['PM2.5'], cmap='YlOrRd', s=60, edgecolor='k', linewidth=0.3)
axes[0].set_title("Sequential Palette (YlOrRd)")
axes[0].set_xlabel("Longitude"); axes[0].set_ylabel("Latitude")
plt.colorbar(sc0, ax=axes[0], label="PM2.5")

diverging_val = df1['PM2.5'] - df1['PM2.5'].mean()
sc1 = axes[1].scatter(df1.lon, df1.lat, c=diverging_val, cmap='RdBu_r', s=60, edgecolor='k', linewidth=0.3)
axes[1].set_title("Diverging Palette (centered on mean)")
axes[1].set_xlabel("Longitude"); axes[1].set_ylabel("Latitude")
plt.colorbar(sc1, ax=axes[1], label="Deviation from mean PM2.5")

df1['Category'] = pd.cut(df1['PM2.5'], bins=[0, 50, 100, 150, 500],
                         labels=['Good', 'Moderate', 'Poor', 'Severe'])
sns.scatterplot(data=df1, x='lon', y='lat', hue='Category', palette='Set1', s=60,
                edgecolor='k', linewidth=0.3, ax=axes[2])
axes[2].set_title("Qualitative Palette (AQI category)")
axes[2].set_xlabel("Longitude"); axes[2].set_ylabel("Latitude")

plt.suptitle("Air Quality Hotspots: Sequential vs Diverging vs Qualitative Color Scales", y=1.03)
plt.tight_layout()
plt.show()
```

Output



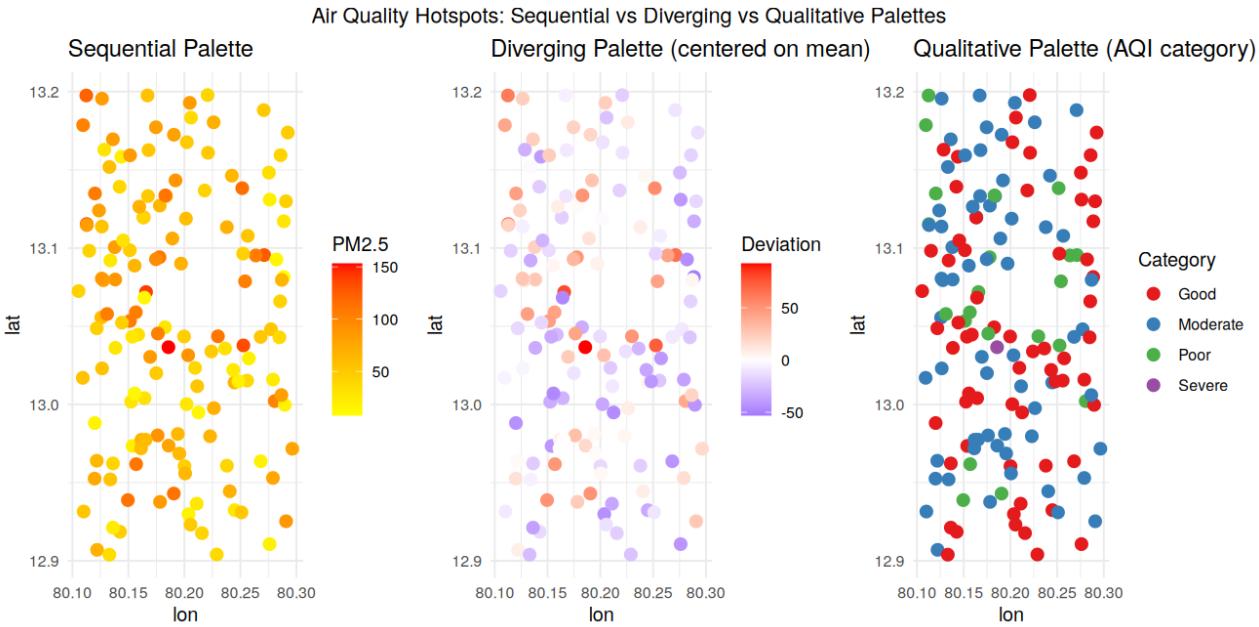
R

Code

```
library(ggplot2)
library(gridExtra)
library(dplyr)

set.seed(1)
n &lt;- 150
```

```
lat &lt;- runif(n, 12.9, 13.2)
lon &lt;- runif(n, 80.1, 80.3)
pm25 &lt;- rgamma(n, shape=4, scale=15)
df1 &lt;- data.frame(lat=lat, lon=lon, PM2.5=pm25)
df1$Deviation &lt;- df1$PM2.5 - mean(df1$PM2.5)
df1$Category &lt;- cut(df1$PM2.5, breaks=c(0,50,100,150,500),
                    labels=c('Good','Moderate','Poor','Severe'))

p1 &lt;- ggplot(df1, aes(x=lon, y=lat, color=PM2.5)) +
  geom_point(size=3) +
  scale_color_gradient(low="yellow", high="red") +
  labs(title="Sequential Palette") + theme_minimal()

p2 &lt;- ggplot(df1, aes(x=lon, y=lat, color=Deviation)) +
  geom_point(size=3) +
  scale_color_gradient2(low="blue", mid="white", high="red", midpoint=0) +
  labs(title="Diverging Palette (centered on mean)") + theme_minimal()

p3 &lt;- ggplot(df1, aes(x=lon, y=lat, color=Category)) +
  geom_point(size=3) +
  scale_color_brewer(palette="Set1") +
  labs(title="Qualitative Palette (AQI category)") + theme_minimal()

grid.arrange(p1, p2, p3, ncol=3,
             top="Air Quality Hotspots: Sequential vs Diverging vs Qualitative Palettes")
```

Output



Scenario 2

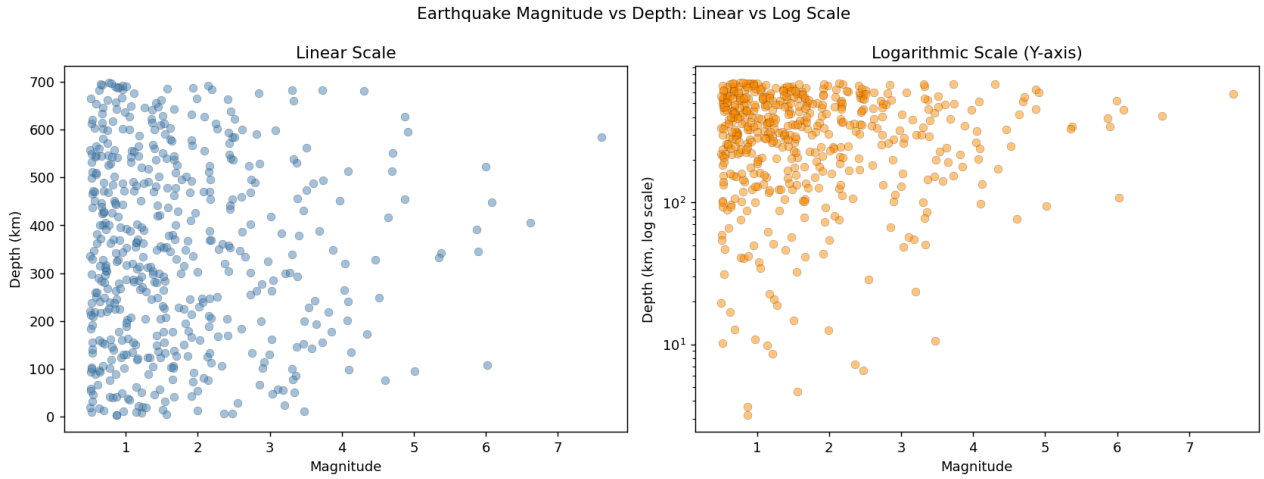
Earthquake Risk Analysis

PYTHON

Code

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)

n = 500
magnitude = np.clip(np.random.exponential(1.2, n) + 0.5, 0.5, 9.5)
depth = np.random.uniform(1, 700, n)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].scatter(magnitude, depth, alpha=0.5, c='steelblue', edgecolor='k', linewidth=0.2)
axes[0].set_title("Linear Scale")
axes[0].set_xlabel("Magnitude"); axes[0].set_ylabel("Depth (km)")

axes[1].scatter(magnitude, depth, alpha=0.5, c='darkorange', edgecolor='k', linewidth=0.2)
axes[1].set_yscale('log')
axes[1].set_title("Logarithmic Scale (Y-axis)")
axes[1].set_xlabel("Magnitude"); axes[1].set_ylabel("Depth (km, log scale)")

plt.suptitle("Earthquake Magnitude vs Depth: Linear vs Log Scale")
plt.tight_layout()
plt.show()
```

Output



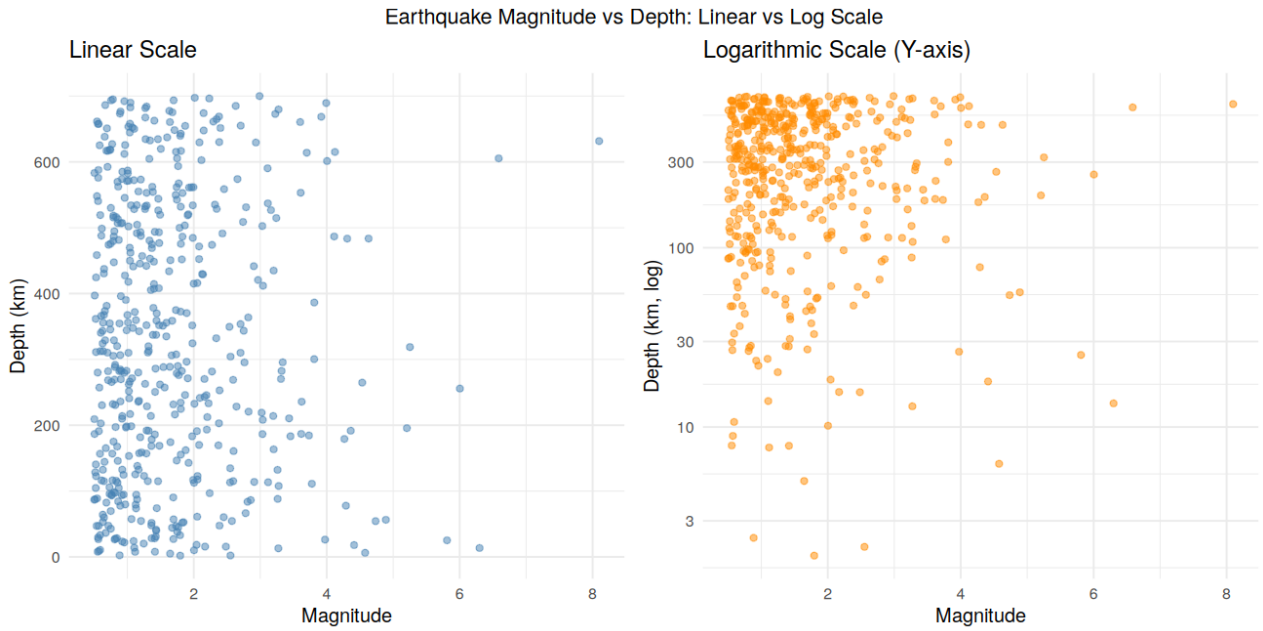
R

Code

```
library(ggplot2)
library(gridExtra)

set.seed(1)
n &lt;- 500
magnitude &lt;- pmin(pmax(rexp(n, rate=1/1.2) + 0.5, 0.5), 9.5)
depth &lt;- runif(n, 1, 700)
df2 &lt;- data.frame(magnitude=magnitude, depth=depth)

p1 &lt;- ggplot(df2, aes(x=magnitude, y=depth)) +
  geom_point(alpha=0.5, color="steelblue") +
  labs(title="Linear Scale", x="Magnitude", y="Depth (km)") +
  theme_minimal()

p2 &lt;- ggplot(df2, aes(x=magnitude, y=depth)) +
```

```
  geom_point(alpha=0.5, color="darkorange") +
  scale_y_log10() +
  labs(title="Logarithmic Scale (Y-axis)", x="Magnitude", y="Depth (km, log)") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2, top="Earthquake Magnitude vs Depth: Linear vs Log Scale")
```

Output



Scenario 3

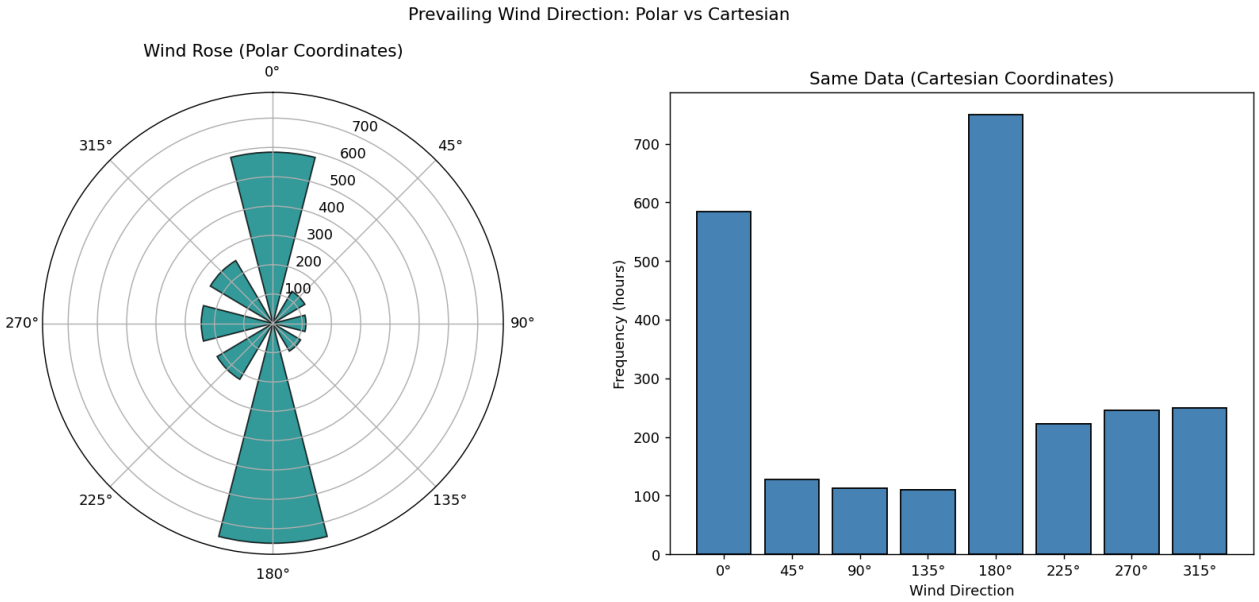
Wind Direction Analysis for Airport Operations

PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(1)

n = 24 * 100
dir_bins = np.arange(0, 360, 45)
directions = np.random.choice(dir_bins, n, p=[0.25, 0.05, 0.05, 0.05, 0.30, 0.10, 0.10, 0.10])
speed = np.random.uniform(2, 25, n)
df3 = pd.DataFrame({'direction': directions, 'speed': speed})

counts = df3.groupby('direction').size().reindex(dir_bins, fill_value=0)
angles = np.radians(dir_bins)

fig = plt.figure(figsize=(13, 6))

ax1 = fig.add_subplot(1, 2, 1, projection='polar')
ax1.bar(angles, counts.values, width=0.5, color='teal', alpha=0.8, edgecolor='k')
ax1.set_theta_zero_location('N')
ax1.set_theta_direction(-1)
ax1.set_title("Wind Rose (Polar Coordinates)")

ax2 = fig.add_subplot(1, 2, 2)
ax2.bar([str(d) + "°" for d in dir_bins], counts.values, color='steelblue', edgecolor='k')
ax2.set_title("Same Data (Cartesian Coordinates)")
ax2.set_xlabel("Wind Direction")
ax2.set_ylabel("Frequency (hours)")

plt.suptitle("Prevailing Wind Direction: Polar vs Cartesian")
plt.tight_layout()
plt.show()
```

Output



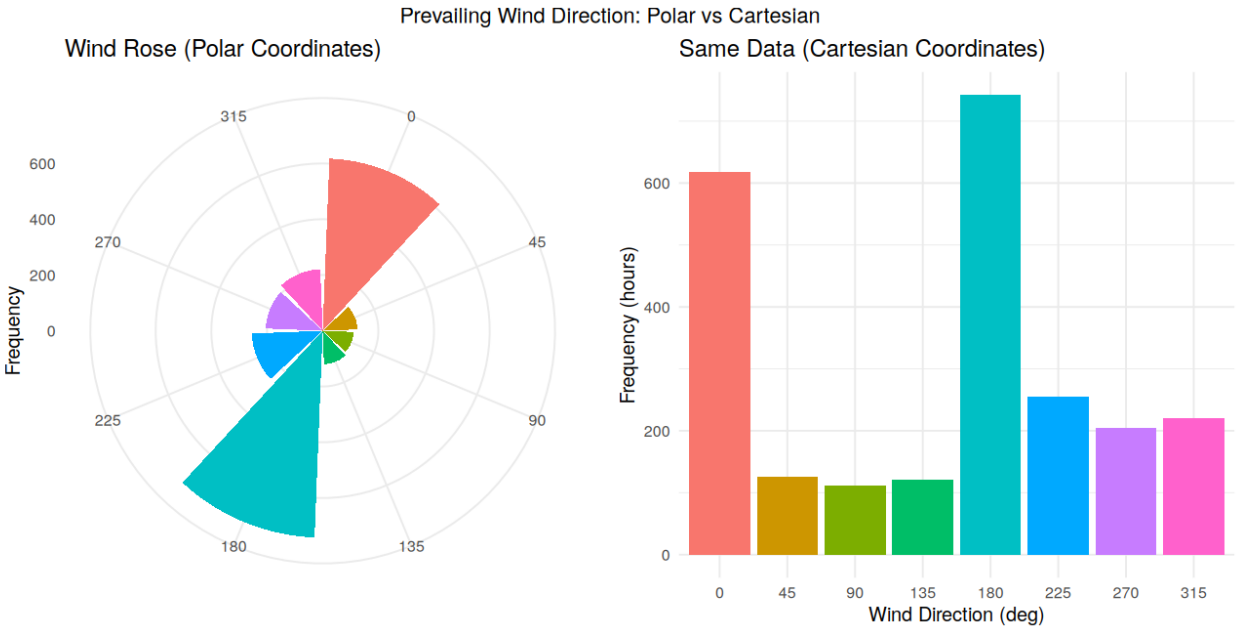
R

Code

```
library(ggplot2)
```

```
library(gridExtra)
library(dplyr)

set.seed(1)
n &lt;- 24 * 100
dir_bins &lt;- seq(0, 315, by=45)
directions &lt;- sample(dir_bins, n, replace=TRUE,
                     prob=c(0.25,0.05,0.05,0.05,0.30,0.10,0.10,0.10))
df3 &lt;- data.frame(direction=directions)

counts &lt;- df3 %&gt;% group_by(direction) %&gt;% summarise(freq=n(), .groups='drop')
counts$direction &lt;- factor(counts$direction, levels=dir_bins)

p1 &lt;- ggplot(counts, aes(x=direction, y=freq, fill=direction)) +
  geom_col(show.legend=FALSE) +
  coord_polar(start=0) +
  labs(title="Wind Rose (Polar Coordinates)", x="", y="Frequency") +
  theme_minimal()

p2 &lt;- ggplot(counts, aes(x=direction, y=freq, fill=direction)) +
  geom_col(show.legend=FALSE) +
  labs(title="Same Data (Cartesian Coordinates)", x="Wind Direction (deg)", y="Frequency (hours)") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2, top="Prevailing Wind Direction: Polar vs Cartesian")
```

Output



Scenario 4

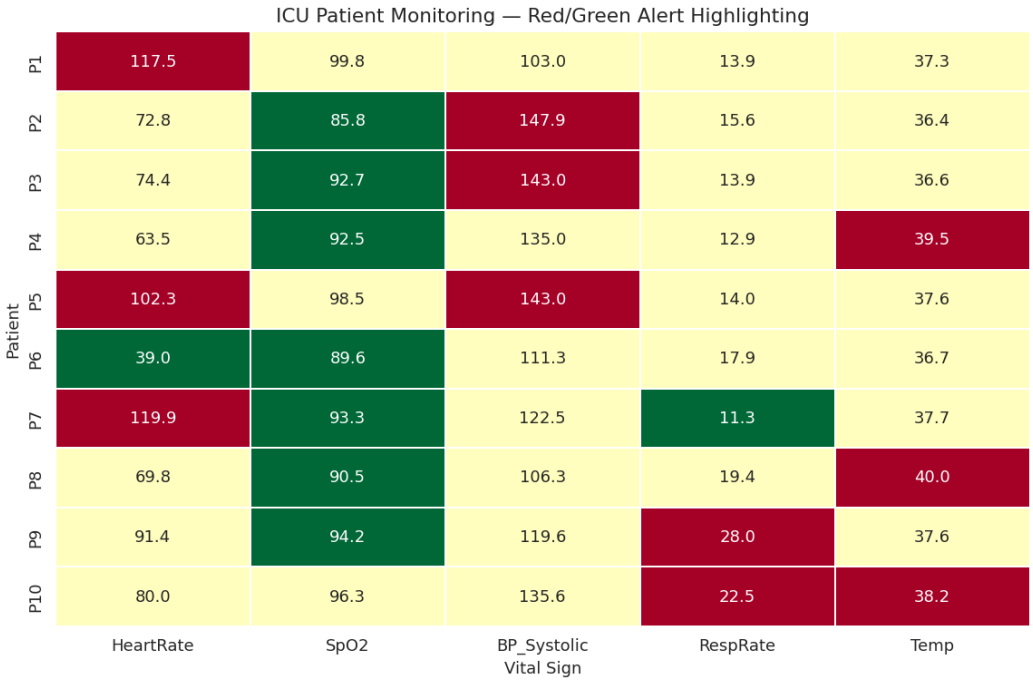
Hospital Patient Monitoring Dashboard

PYTHON

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_style("white")
np.random.seed(1)

patients = [f"P{i+1}" for i in range(10)]
vitals = ['HeartRate', 'SpO2', 'BP_Systolic', 'RespRate', 'Temp']

data = {
    'HeartRate': np.random.normal(85, 20, 10),
    'SpO2': np.random.normal(94, 4, 10),
    'BP_Systolic': np.random.normal(125, 20, 10),
    'RespRate': np.random.normal(18, 6, 10),
    'Temp': np.random.normal(37.5, 1.2, 10),
}
df4 = pd.DataFrame(data, index=patients)

# normal ranges: (low, high) -&gt; outside range flagged
thresholds = {
    'HeartRate': (60, 100),
    'SpO2': (95, 100),
    'BP_Systolic': (90, 140),
    'RespRate': (12, 20),
    'Temp': (36.1, 37.8),
}

status = pd.DataFrame(index=patients, columns=vitals)
for v in vitals:
    low, high = thresholds[v]
    status[v] = np.where(df4[v] &lt; low, 'Low', np.where(df4[v] &gt; high, 'High', 'Normal'))

color_map = {'Normal': 0, 'Low': -1, 'High': 1}
color_grid = status.replace(color_map).astype(int)

plt.figure(figsize=(9, 6))
sns.heatmap(color_grid, cmap='RdYlGn_r', center=0, annot=df4.round(1), fmt='',
            cbar=False, linewidths=1, linecolor='white')
plt.title("ICU Patient Monitoring — Red/Green Alert Highlighting")
plt.xlabel("Vital Sign"); plt.ylabel("Patient")
plt.tight_layout()
plt.show()
```

Output



R

Code

```
library(ggplot2)
library(reshape2)

set.seed(1)
patients &lt;- paste0("P", 1:10)
vitals &lt;- c('HeartRate','SpO2','BP_Systolic','RespRate','Temp')

df4 &lt;- data.frame(
  Patient = patients,
  HeartRate = rnorm(10, 85, 20),
  SpO2 = rnorm(10, 94, 4),
  BP_Systolic = rnorm(10, 125, 20),
  RespRate = rnorm(10, 18, 6),
  Temp = rnorm(10, 37.5, 1.2)
)

thresholds &lt;- list(
  HeartRate = c(60, 100),
  SpO2 = c(95, 100),
  BP_Systolic = c(90, 140),
  RespRate = c(12, 20),
  Temp = c(36.1, 37.8)
)

status &lt;- df4
for (v in vitals) {
  low &lt;- thresholds[[v]][1]; high &lt;- thresholds[[v]][2]
  status[[v]] &lt;- ifelse(df4[[v]] &lt; low, -1, ifelse(df4[[v]] &gt; high, 1, 0))
}

long_status &lt;- melt(status, id.vars="Patient", variable.name="Vital", value.name="AlertLevel")
long_vals &lt;- melt(df4, id.vars="Patient", variable.name="Vital", value.name="Value")
long_status$Label &lt;- round(long_vals$Value, 1)

ggplot(long_status, aes(x=Vital, y=Patient, fill=AlertLevel)) +
  geom_tile(color="white") +
  geom_text(aes(label=Label), size=3.3) +
  scale_fill_gradient2(low="red", mid="green", high="red", midpoint=0, guide="none") +
  labs(title="ICU Patient Monitoring - Red/Green Alert Highlighting") +
  theme_minimal()
```

Output


ICU Patient Monitoring - Red/Green Alert Highlighting
Patient
P9 96.5 97.3 115.4 24.6 37.4
P8 99.8 97.8 95.6 17.6 38.4
P7 94.7 93.9 121.9 15.6 37.9
P6 68.6 93.8 123.9 15.5 36.7
P5 91.6 98.5 137.4 9.7 36.7
P4 116.9 85.1 85.2 17.7 38.2
P3 68.3 91.5 126.5 20.3 38.3
P2 88.7 95.6 140.6 17.4 37.2
P10 78.9 96.4 133.4 22.6 38.6
P1 72.5 100 143.4 26.2 37.3
HeartRate
SpO2
BP_Systolic
RespRate
Temp
Vital

Scenario 5

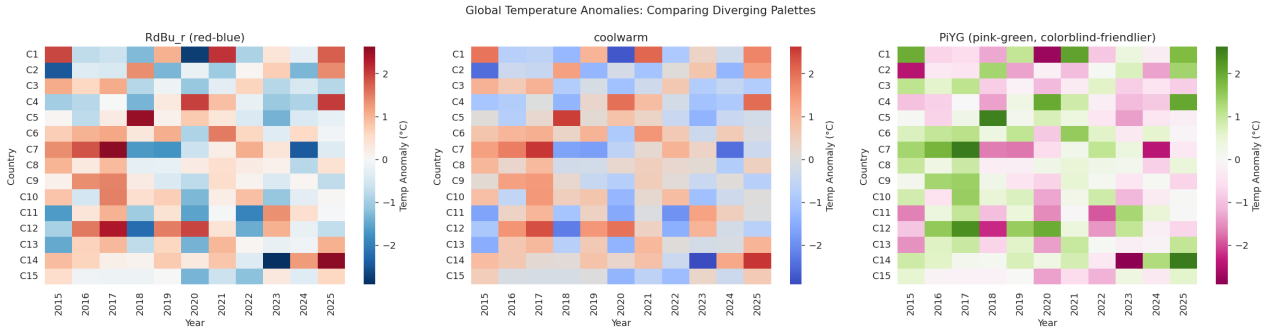
Global Climate Change Study

PYTHON

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_style("white")
np.random.seed(1)

countries = [f"C{i+1}" for i in range(15)]
years = list(range(2015, 2026))
anomaly = np.random.normal(0, 1.2, (len(countries), len(years)))
df5 = pd.DataFrame(anomaly, index=countries, columns=years)

palettes = ['RdBu_r', 'coolwarm', 'PiYG']
titles = ['RdBu_r (red-blue)', 'coolwarm', 'PiYG (pink-green, colorblind-friendlier)']

fig, axes = plt.subplots(1, 3, figsize=(19, 5))
for ax, pal, title in zip(axes, palettes, titles):
    sns.heatmap(df5, cmap=pal, center=0, ax=ax, cbar_kws={'label': 'Temp Anomaly (°C)'})
    ax.set_title(title)
    ax.set_xlabel("Year"); ax.set_ylabel("Country")

plt.suptitle("Global Temperature Anomalies: Comparing Diverging Palettes")
plt.tight_layout()
plt.show()
```

Output



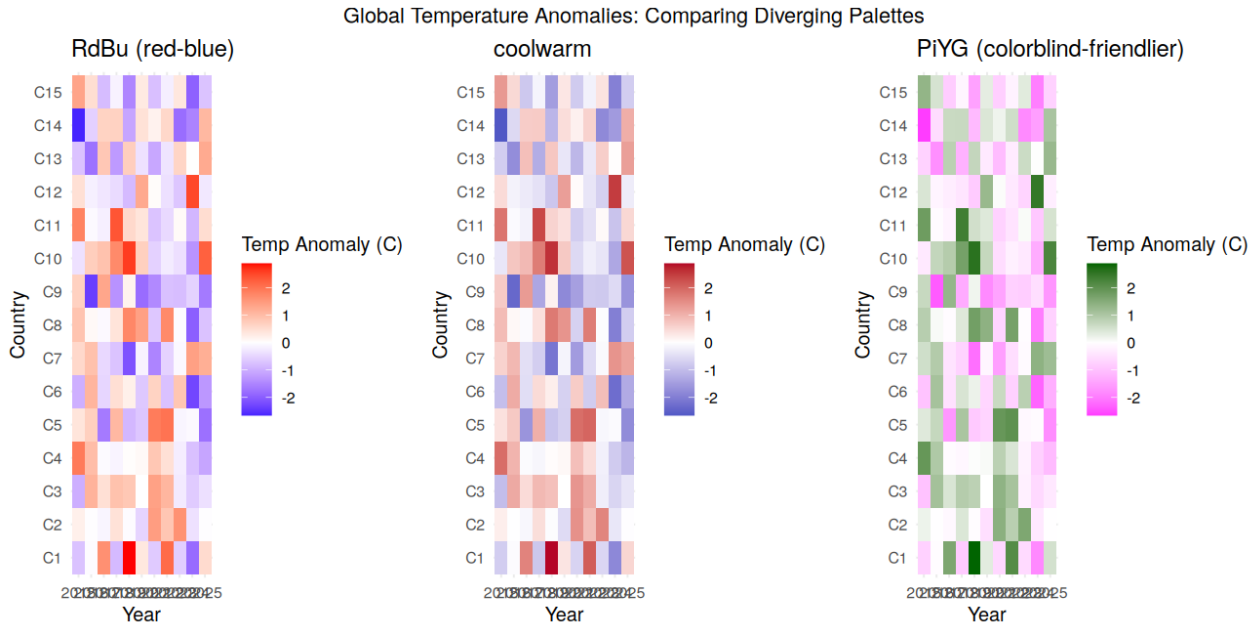
R

Code

```
library(ggplot2)
library(gridExtra)
library(reshape2)

set.seed(1)
countries &lt;- paste0("C", 1:15)
years &lt;- 2015:2025
anomaly &lt;- matrix(rnorm(length(countries)*length(years), 0, 1.2),
                  nrow=length(countries))
rownames(anomaly) &lt;- countries
colnames(anomaly) &lt;- years

df5 &lt;- melt(anomaly)
colnames(df5) &lt;- c("Country", "Year", "Anomaly")

make_plot &lt;- function(low, high, title) {
  ggplot(df5, aes(x=factor(Year), y=Country, fill=Anomaly)) +
    geom_tile() +
    scale_fill_gradient2(low=low, mid="white", high=high, midpoint=0,
                         name="Temp Anomaly (C)") +
```

```
    labs(title=title, x="Year", y="Country") +
    theme_minimal()
}

p1 &lt;- make_plot("blue", "red", "RdBu (red-blue)")
p2 &lt;- make_plot("#3B4CC0", "#B40426", "coolwarm")
p3 &lt;- make_plot("magenta", "darkgreen", "PiYG (colorblind-friendlier)")

grid.arrange(p1, p2, p3, ncol=3,
             top="Global Temperature Anomalies: Comparing Diverging Palettes")
```

Output



Scenario 6

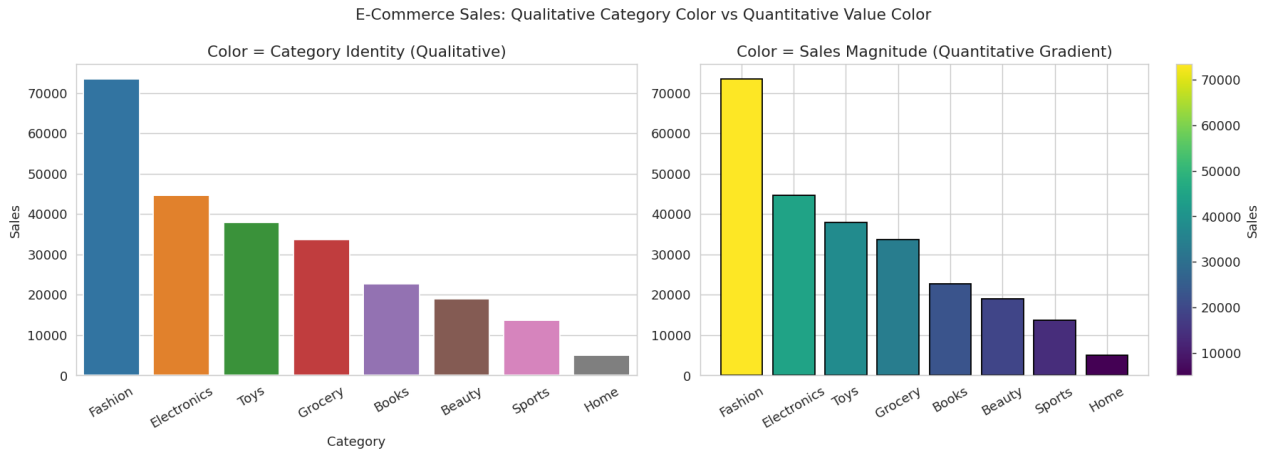
E-Commerce Sales Analytics

PYTHON

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_style("whitegrid")
np.random.seed(1)

categories = ['Electronics', 'Fashion', 'Home', 'Grocery', 'Beauty', 'Sports', 'Books', 'Toys']
sales = np.random.uniform(5000, 100000, len(categories))
df6 = pd.DataFrame({'Category': categories, 'Sales': sales}).sort_values('Sales', ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# qualitative color per category (distinguish identity)
sns.barplot(data=df6, x='Category', y='Sales', hue='Category', palette='tab10', legend=False, ax=axes[0])
axes[0].set_title("Color = Category Identity (Qualitative)")
axes[0].tick_params(axis='x', rotation=30)

# quantitative color gradient on same bars (represent magnitude)
norm = plt.Normalize(df6.Sales.min(), df6.Sales.max())
colors = plt.cm.viridis(norm(df6.Sales.values))
axes[1].bar(df6.Category, df6.Sales, color=colors, edgecolor='k')
axes[1].tick_params(axis='x', rotation=30)
axes[1].set_title("Color = Sales Magnitude (Quantitative Gradient)")
sm = plt.cm.ScalarMappable(cmap='viridis', norm=norm)
plt.colorbar(sm, ax=axes[1], label='Sales')

plt.suptitle("E-Commerce Sales: Qualitative Category Color vs Quantitative Value Color")
plt.tight_layout()
plt.show()
```

Output



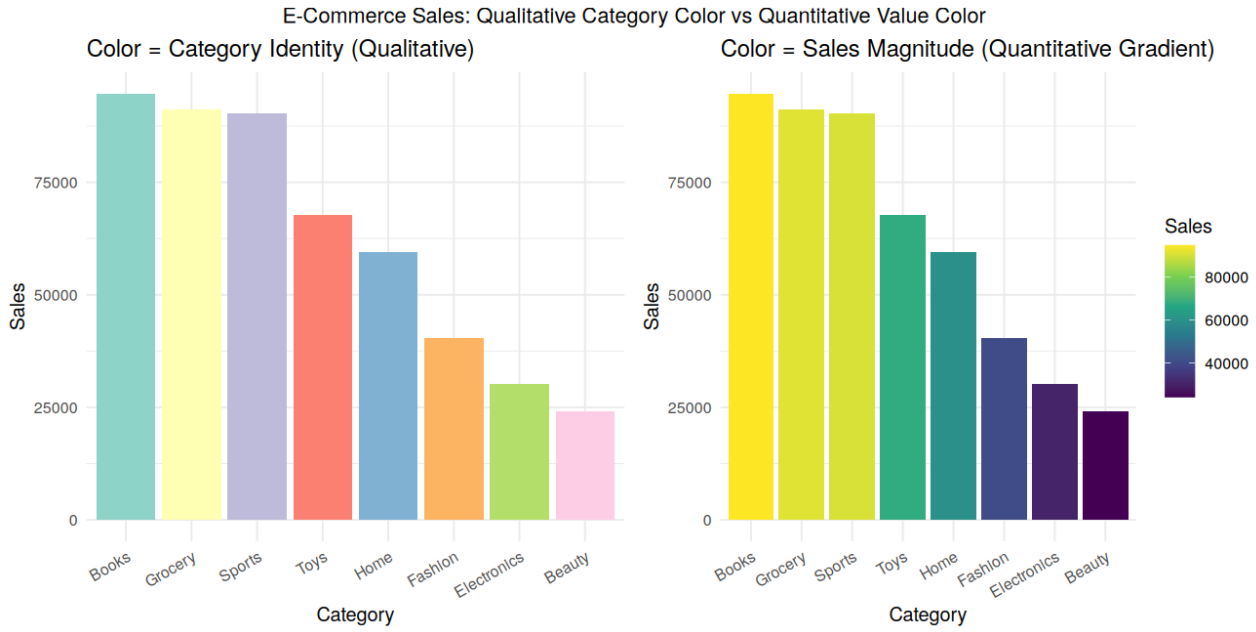
R

Code

```
library(ggplot2)
library(gridExtra)
library(dplyr)

set.seed(1)
categories &lt;- c('Electronics','Fashion','Home','Grocery','Beauty','Sports','Books','Toys')
sales &lt;- runif(length(categories), 5000, 100000)
df6 &lt;- data.frame(Category=categories, Sales=sales) %&gt;% arrange(desc(Sales))
df6$Category &lt;- factor(df6$Category, levels=df6$Category)
```

```
p1 &lt;- ggplot(df6, aes(x=Category, y=Sales, fill=Category)) +
  geom_col(show.legend=FALSE) +
  scale_fill_brewer(palette="Set3") +
  labs(title="Color = Category Identity (Qualitative)") +
  theme_minimal() + theme(axis.text.x=element_text(angle=30, hjust=1))

p2 &lt;- ggplot(df6, aes(x=Category, y=Sales, fill=Sales)) +
  geom_col() +
  scale_fill_viridis_c() +
  labs(title="Color = Sales Magnitude (Quantitative Gradient)") +
  theme_minimal() + theme(axis.text.x=element_text(angle=30, hjust=1))

grid.arrange(p1, p2, ncol=2,
             top="E-Commerce Sales: Qualitative Category Color vs Quantitative Value Color")
```

Output



Scenario 7

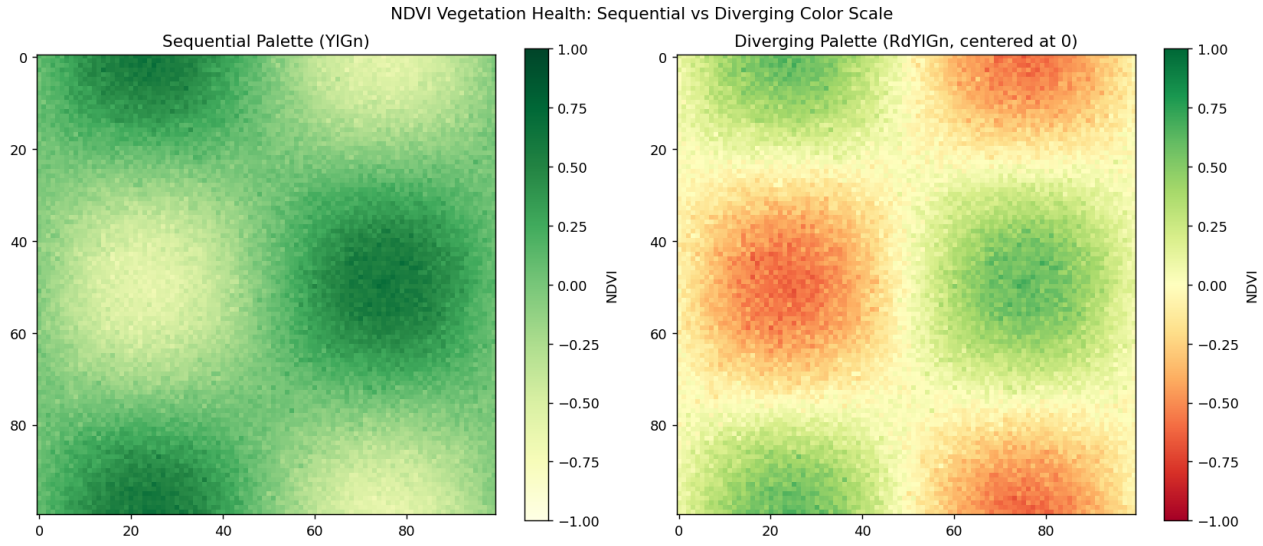
Satellite Image Vegetation Analysis (NDVI)

PYTHON

Code

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)

size = 100
x = np.linspace(-3, 3, size)
y = np.linspace(-3, 3, size)
xx, yy = np.meshgrid(x, y)
ndvi = np.sin(xx) * np.cos(yy) * 0.6 + np.random.normal(0, 0.05, (size, size))
ndvi = np.clip(ndvi, -1, 1)

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

im0 = axes[0].imshow(ndvi, cmap='YlGn', vmin=-1, vmax=1)
axes[0].set_title("Sequential Palette (YlGn)")
plt.colorbar(im0, ax=axes[0], label='NDVI')

im1 = axes[1].imshow(ndvi, cmap='RdYlGn', vmin=-1, vmax=1)
axes[1].set_title("Diverging Palette (RdYlGn, centered at 0)")
plt.colorbar(im1, ax=axes[1], label='NDVI')

plt.suptitle("NDVI Vegetation Health: Sequential vs Diverging Color Scale")
plt.tight_layout()
plt.show()
```

Output



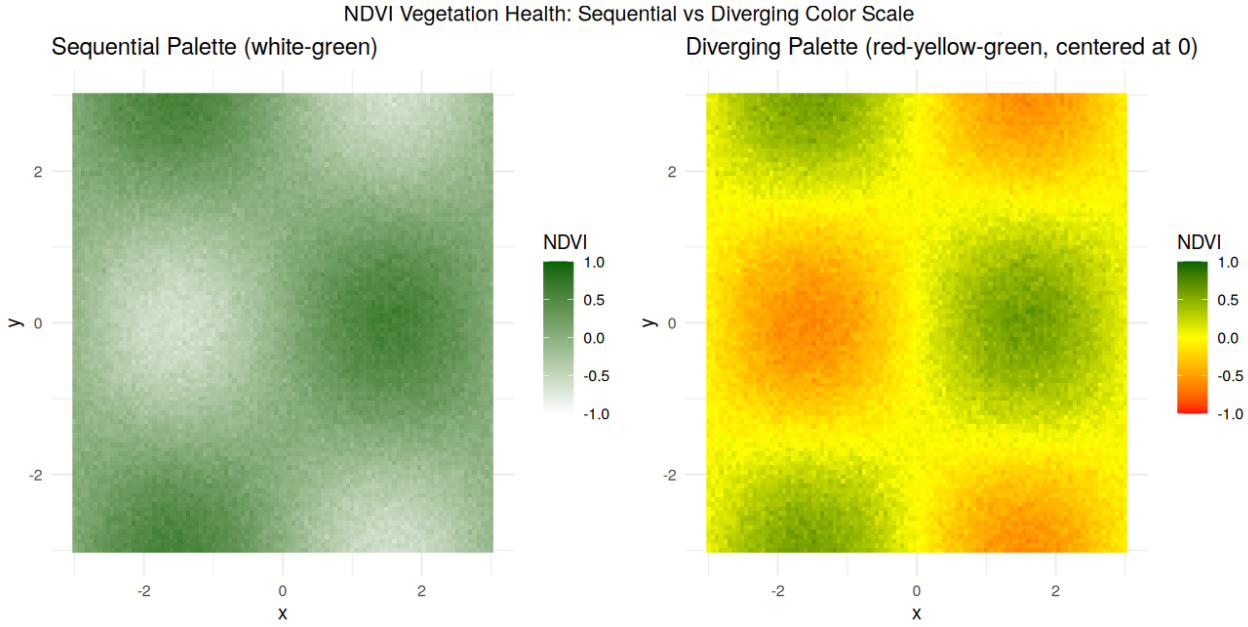
R

Code

```
library(ggplot2)
library(gridExtra)

set.seed(1)
size &lt;- 100
x &lt;- seq(-3, 3, length.out=size)
y &lt;- seq(-3, 3, length.out=size)
grid &lt;- expand.grid(x=x, y=y)
grid$ndvi &lt;- sin(grid$x) * cos(grid$y) * 0.6 + rnorm(nrow(grid), 0, 0.05)
grid$ndvi &lt;- pmin(pmax(grid$ndvi, -1), 1)
p1 &lt;- ggplot(grid, aes(x=x, y=y, fill=ndvi)) +
```

```
  geom_raster() +
  scale_fill_gradient(low="white", high="darkgreen", limits=c(-1,1), name="NDVI") +
  labs(title="Sequential Palette (white-green)") +
  theme_minimal()

p2 &lt;- ggplot(grid, aes(x=x, y=y, fill=ndvi)) +
  geom_raster() +
  scale_fill_gradient2(low="red", mid="yellow", high="darkgreen", midpoint=0,
                       limits=c(-1,1), name="NDVI") +
  labs(title="Diverging Palette (red-yellow-green, centered at 0)") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2,
             top="NDVI Vegetation Health: Sequential vs Diverging Color Scale")
```

Output



Scenario 8

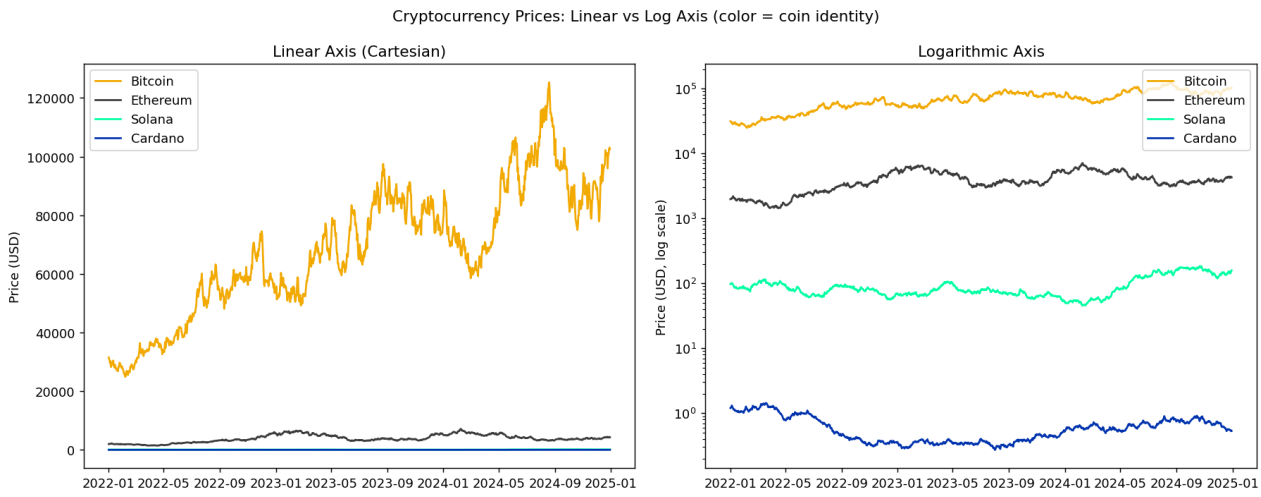
Cryptocurrency Market Volatility

PYTHON

Code

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(1)

days = pd.date_range("2022-01-01", periods=365 * 3, freq='D')
coins = {'Bitcoin': 30000, 'Ethereum': 2000, 'Solana': 100, 'Cardano': 1.2}
colors = {'Bitcoin': '#f2a900', 'Ethereum': '#3c3c3d', 'Solana': '#00ffa3', 'Cardano': '#0033ad'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

df8 = pd.DataFrame(index=days)
for coin, start in coins.items():
    walk = start * np.exp(np.cumsum(np.random.normal(0, 0.03, len(days))))
    df8[coin] = walk
    axes[0].plot(days, df8[coin], color=colors[coin], label=coin)
    axes[1].plot(days, df8[coin], color=colors[coin], label=coin)

axes[0].set_title("Linear Axis (Cartesian)")
axes[0].set_ylabel("Price (USD)")
axes[0].legend()

axes[1].set_yscale('log')
axes[1].set_title("Logarithmic Axis")
axes[1].set_ylabel("Price (USD, log scale)")
axes[1].legend()

plt.suptitle("Cryptocurrency Prices: Linear vs Log Axis (color = coin identity)")
plt.tight_layout()
plt.show()
```

Output



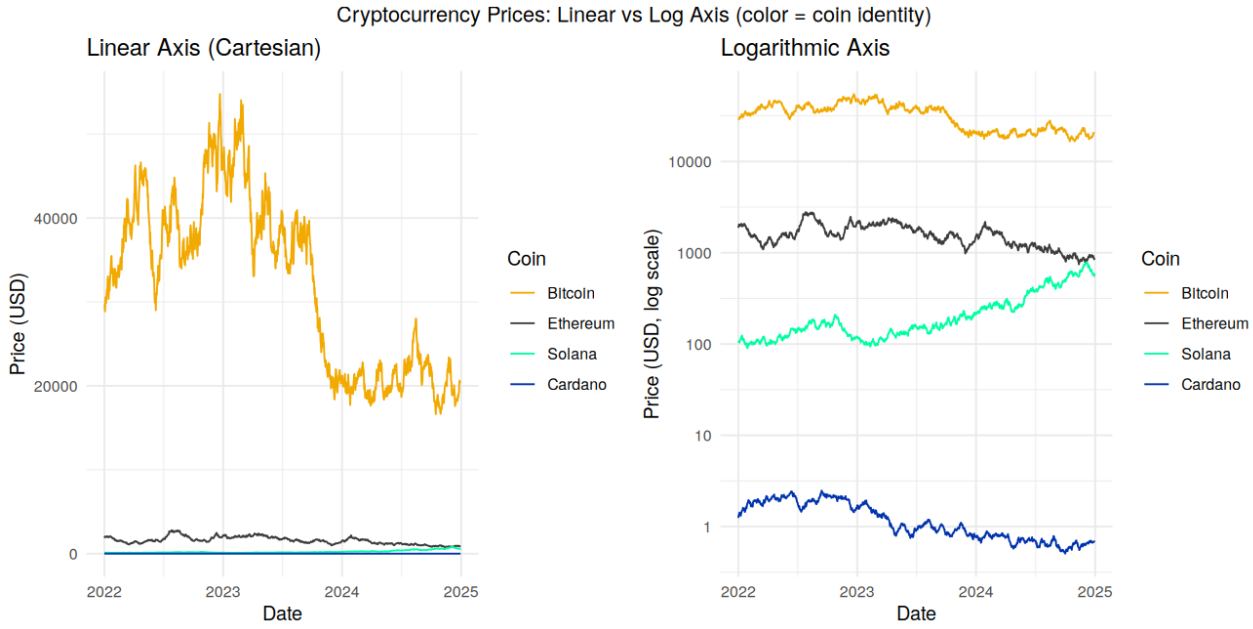
R

Code

```
library(ggplot2)
library(gridExtra)

set.seed(1)
days &lt;- seq(as.Date("2022-01-01"), by="day", length.out=365*3)
coins &lt;- list(Bitcoin=30000, Ethereum=2000, Solana=100, Cardano=1.2)
coin_colors &lt;- c(Bitcoin="#f2a900", Ethereum="#3c3c3d", Solana="#00ffa3", Cardano="#0033ad")
```

```
df8 &lt;- data.frame()
for (coin in names(coins)) {
  walk &lt;- coins[[coin]] * exp(cumsum(rnorm(length(days), 0, 0.03)))
  df8 &lt;- rbind(df8, data.frame(Date=days, Price=walk, Coin=coin))
}
df8$Coin &lt;- factor(df8$Coin, levels=names(coins))

p1 &lt;- ggplot(df8, aes(x=Date, y=Price, color=Coin)) +
  geom_line() +
  scale_color_manual(values=coin_colors) +
  labs(title="Linear Axis (Cartesian)", y="Price (USD)") +
  theme_minimal()

p2 &lt;- ggplot(df8, aes(x=Date, y=Price, color=Coin)) +
  geom_line() +
  scale_color_manual(values=coin_colors) +
  scale_y_log10() +
  labs(title="Logarithmic Axis", y="Price (USD, log scale)") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2,
             top="Cryptocurrency Prices: Linear vs Log Axis (color = coin identity)")
```

Output



Scenario 9

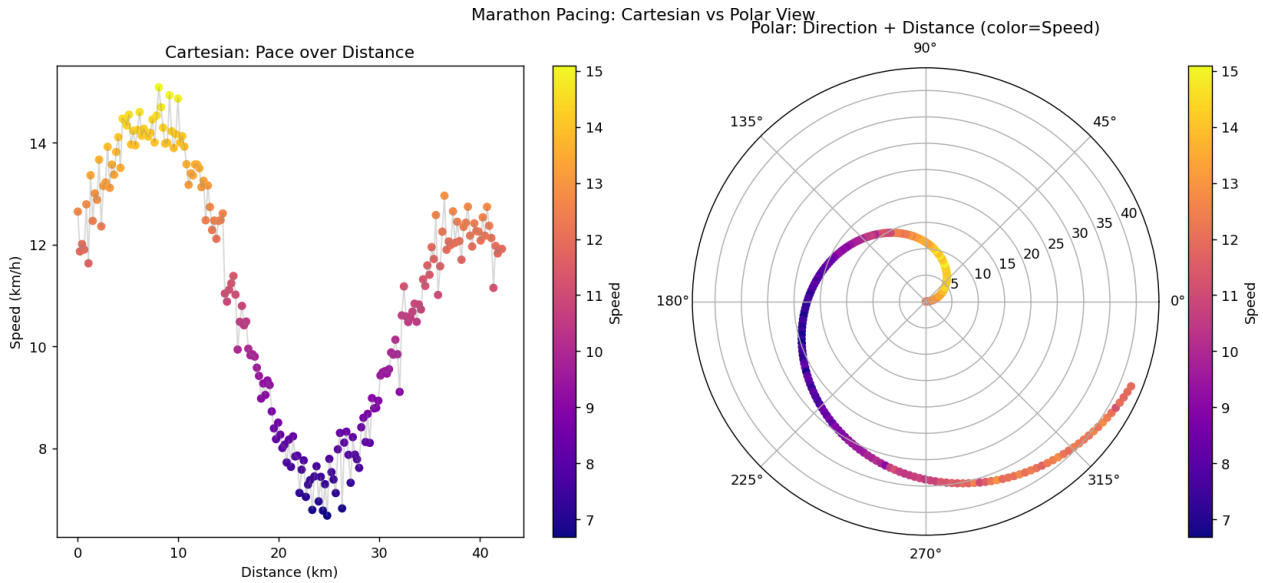
Marathon Runner Performance Analysis

PYTHON

Code

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)

n = 200
distance = np.linspace(0, 42.2, n)
speed = 12 + 3 * np.sin(distance / 5) - distance / 42.2 * 3 + np.random.normal(0, 0.4, n)
direction = (distance * 8) % 360  # simulate a winding race course

fig = plt.figure(figsize=(13, 6))

ax1 = fig.add_subplot(1, 2, 1)
sc1 = ax1.scatter(distance, speed, c=speed, cmap='plasma', s=25)
ax1.plot(distance, speed, color='gray', alpha=0.3, linewidth=1)
ax1.set_title("Cartesian: Pace over Distance")
ax1.set_xlabel("Distance (km)"); ax1.set_ylabel("Speed (km/h)")
plt.colorbar(sc1, ax=ax1, label="Speed")

ax2 = fig.add_subplot(1, 2, 2, projection='polar')
theta = np.radians(direction)
sc2 = ax2.scatter(theta, distance, c=speed, cmap='plasma', s=25)
ax2.set_title("Polar: Direction + Distance (color=Speed)")
plt.colorbar(sc2, ax=ax2, label="Speed")

plt.suptitle("Marathon Pacing: Cartesian vs Polar View")
plt.tight_layout()
plt.show()
```

Output



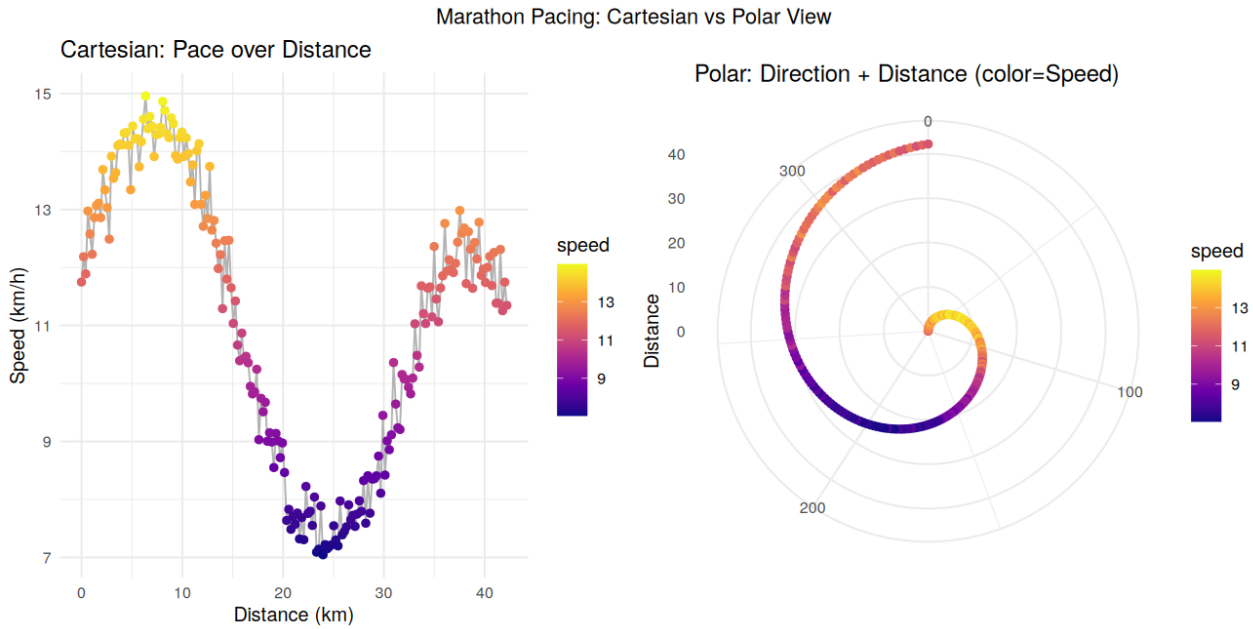
R

Code

```
library(ggplot2)
library(gridExtra)

set.seed(1)
n &lt;- 200
distance &lt;- seq(0, 42.2, length.out=n)
```

```
speed &lt;- 12 + 3*sin(distance/5) - distance/42.2*3 + rnorm(n, 0, 0.4)
direction &lt;- (distance * 8) %% 360
df9 &lt;- data.frame(distance=distance, speed=speed, direction=direction)

p1 &lt;- ggplot(df9, aes(x=distance, y=speed, color=speed)) +
  geom_line(color="gray70") +
  geom_point(size=1.8) +
  scale_color_viridis_c(option="plasma") +
  labs(title="Cartesian: Pace over Distance", x="Distance (km)", y="Speed (km/h)") +
  theme_minimal()

p2 &lt;- ggplot(df9, aes(x=direction, y=distance, color=speed)) +
  geom_point(size=1.8) +
  coord_polar(theta="x") +
  scale_color_viridis_c(option="plasma") +
  labs(title="Polar: Direction + Distance (color=Speed)", x="", y="Distance") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2, top="Marathon Pacing: Cartesian vs Polar View")
```

Output



Scenario 10

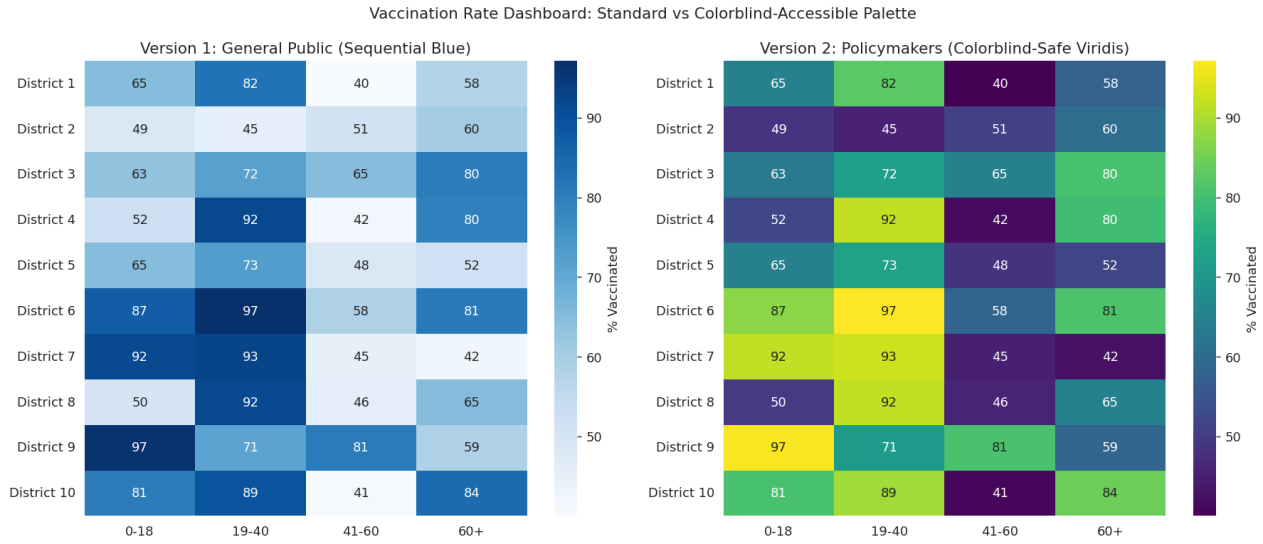
COVID-19 Vaccination Campaign Dashboard

PYTHON

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

sns.set_style("white")
np.random.seed(1)

districts = [f"District {i+1}" for i in range(10)]
age_groups = ['0-18', '19-40', '41-60', '60+']
vax_rate = np.random.uniform(40, 99, (len(districts), len(age_groups)))
df10 = pd.DataFrame(vax_rate, index=districts, columns=age_groups)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Version 1: standard sequential palette (general public - simple, high contrast)
sns.heatmap(df10, cmap='Blues', annot=True, fmt='.0f', ax=axes[0], cbar_kws={'label': '% Vaccinated'})
axes[0].set_title("Version 1: General Public (Sequential Blue)")

# Version 2: colorblind-safe palette (policymakers - viridis is deuteranopia/protanopia safe)
sns.heatmap(df10, cmap='viridis', annot=True, fmt='.0f', ax=axes[1], cbar_kws={'label': '% Vaccinated'})
axes[1].set_title("Version 2: Policymakers (Colorblind-Safe Viridis)")

plt.suptitle("Vaccination Rate Dashboard: Standard vs Colorblind-Accessible Palette")
plt.tight_layout()
plt.show()
```

Output



R

Code

```
library(ggplot2)
library(gridExtra)
library(reshape2)

set.seed(1)
districts &lt;- paste0("District ", 1:10)
age_groups &lt;- c('0-18','19-40','41-60','60+')
vax_rate &lt;- matrix(runif(length(districts)*length(age_groups), 40, 99),
                   nrow=length(districts))
rownames(vax_rate) &lt;- districts
```

```
colnames(vax_rate) &lt;- age_groups

df10 &lt;- melt(vax_rate)
colnames(df10) &lt;- c("District", "AgeGroup", "Rate")

p1 &lt;- ggplot(df10, aes(x=AgeGroup, y=District, fill=Rate)) +
  geom_tile() +
  geom_text(aes(label=round(Rate)), size=3) +
  scale_fill_gradient(low="white", high="#08306b", name="% Vaccinated") +
  labs(title="Version 1: General Public (Sequential Blue)") +
  theme_minimal()

p2 &lt;- ggplot(df10, aes(x=AgeGroup, y=District, fill=Rate)) +
  geom_tile() +
  geom_text(aes(label=round(Rate)), size=3, color="white") +
  scale_fill_viridis_c(name="% Vaccinated") +
  labs(title="Version 2: Policymakers (Colorblind-Safe Viridis)") +
  theme_minimal()

grid.arrange(p1, p2, ncol=2,
             top="Vaccination Rate Dashboard: Standard vs Colorblind-Accessible Palette")
```

Output